SEELE AI — 10-PHASE EXECUTION DIRECTIVE

Final Evolution Lab: Complete Implementation Pass

Production Specification v1.0 · Compilation Target

> **Repository:** `/home/ubuntu/github_repos/Final-Evolution-Lab/`
> **Branch:** `anti-gravity-fel` (commit 9519541)
> **Engine:** Unreal Engine 5.7
> **Platform:** iOS Shipping (native) + Linux E3DS
> **Architecture:** UE 5.7 native host → Swift shell → WKWebView overlay → FastAPI backend → MongoDB/Firestore
> **Design Sources:** `fel_master_engine_brief_design.md`, `fel_per_game_mode_blueprint_design.md`, `fel_master_architecture_blueprint_design.md`

EXECUTION RULES

1. **Each phase MUST complete before starting the next.** No skipping.
2. **Each phase ends with a QUALITY GATE.** If any gate check fails, fix before proceeding.
3. **Do not invent environment layouts, venue names, or map paths.** Use only what exists in `[EmergentPlayMap]` section of `infra/ue5_config/DefaultGame.ini` as source of truth.
4. **Cooked iOS path format is `/Game/FEL/Venues/{VenueName}/{VenueName}`.** The `/Game/FEL/Maps/` prefix in `FEL_ModeManager.production.json` is WRONG and must be corrected.
5. **`karate` is a private alias for `karate_h2h`.** Do not expose it as a public mode.
6. **`mario_party_fever` is renamed to `court_carnival`.** All references must be updated.
7. **`market_browse` is a Module Library, not a scoring game mode.** No session receipt, no PRQ, no shards.
8. **`who_scene_it` and `court_carnival` MUST NOT be marked production** until routing, gameplay contracts, and session receipts are complete. They are **preview**.
9. **Do not introduce Unity, SwiftUI-first shell, or Unreal-as-a-Library patterns.** UE 5.7 is the native host. Swift shell wraps it. WKWebView overlay sits inside UE.
10. **Test after every phase.** Run existing tests + the phase-specific quality gate.

PHASE 1: REGISTRY ALIGNMENT

Goal: Make all 6 mode registries agree on the same 19 modes with correct statuses, paths, and IDs.

1.1 Fix `backend/FEL_ModeManager.production.json`

- Change `"total_modes": 17` → `"total_modes": 19`
- Change `"production_modes": 12` → `"production_modes": 12` (keep — the 12 actual production modes)
- Fix ALL `"map"` values: replace `/Game/FEL/Maps/` prefix with correct `/Game/FEL/Venues/{Token}/{Token}` paths:

```
basketball_h2h:   /Game/FEL/Venues/VeniceBeach/VeniceBeach
  basketball_dunk:  /Game/FEL/Venues/VeniceBeach/VeniceBeach
  basketball_3v3:   /Game/FEL/Venues/VeniceBeach/VeniceBeach
  karate_h2h:       /Game/FEL/Venues/Dojo/Dojo
  karate_endless:   /Game/FEL/Venues/Dojo/Dojo
  baseball:         /Game/FEL/Venues/BaseballPark/BaseballPark
  football:         /Game/FEL/Venues/Gridiron/Gridiron
  soccer:           /Game/FEL/Venues/SoccerStadium/SoccerStadium
  golf:             /Game/FEL/Venues/Links/Links
  tennis:           /Game/FEL/Venues/TennisCourt/TennisCourt
  volleyball:       /Game/FEL/Venues/SandCourt/SandCourt
  surfing:          /Game/FEL/Venues/VeniceBeach/VeniceBeach
  skateboarding:    /Game/FEL/Venues/Skate_Park/Skate_Park
  snowboarding:     /Game/FEL/Venues/Mountain_Slope/Mountain_Slope
  gymnastics:       /Game/FEL/Venues/TrainingFloor/TrainingFloor
  brain_brawl:      /Game/FEL/Venues/NeuroArena/NeuroArena
  market_browse:    /Game/FEL/Venues/Luma_Venice_Shop/Luma_Venice_Shop
  who_scene_it:     /Game/FEL/Venues/NeuroArena/NeuroArena
  court_carnival:   /Game/FEL/Venues/VeniceBeach/VeniceBeach
```

- Change `who_scene_it` status: `"production"` → `"preview"`
- Rename key `mario_party_fever` → `court_carnival`, set status: `"preview"`
- Update `gamemode_class` for court_carnival: `/Script/FEL.BP_CourtCarnival`

1.2 Fix `FinalEvolutionLab/Models/GameMode.swift`

- Add 3 new enum cases to `GameModeId`:

```swift
  case whoSceneIt = "who_scene_it"
  case courtCarnival = "court_carnival"
  case marketBrowse = "market_browse"
```

- Add `inputScheme` mappings:

```swift
  case .whoSceneIt, .courtCarnival: return .dragTap
  case .marketBrowse: return .dragTap
```

- Add 3 entries to `GameModeRegistry.all`:

```swift
  GameMode(id: .whoSceneIt, name: "Who Scene It", subtitle: "Scene Recognition",
           sport: .academy, iconName: "eye.fill",
           accentColor: Color(red: 0.55, green: 0.35, blue: 1.0),
           multiplayerType: .realtime, environmentName: "Neuro Arena", hint: nil)
  GameMode(id: .courtCarnival, name: "Court Carnival", subtitle: "Party Arcade",
           sport: .field, iconName: "party.popper",
           accentColor: Color(red: 1.0, green: 0.8, blue: 0.0),
           multiplayerType: .realtime, environmentName: "Venice Beach Court", hint: nil)
  GameMode(id: .marketBrowse, name: "Module Library", subtitle: "Browse & Collect",
           sport: .academy, iconName: "storefront",
           accentColor: Color(red: 0.0, green: 0.9, blue: 0.7),
           multiplayerType: .solo, environmentName: "Luma Venice Shop", hint: nil)
```

1.3 Fix `backend/ue_mode_maps.json`

- Add 3 entries to `mode_to_unreal_map`:

```
json
  "who_scene_it": "Neuro_Arena",
  "court_carnival": "Venice_Beach_Court",
  "market_browse": "Sovereign_Shop"
```

1.4 Fix `infra/ue5_config/DefaultGame.ini`

- In `[EmergentPlayMap]` section:
- REMOVE: `skateboarding=/Game/FEL/Venues/VeniceBeach/VeniceBeach` (wrong venue — routes to basketball court instead of skate park)
- REMOVE: `snowboarding=/Game/FEL/Venues/VeniceBeach/VeniceBeach` (wrong venue — routes to beach instead of mountain)
- ADD: `who_scene_it=/Game/FEL/Venues/NeuroArena/NeuroArena`
- ADD: `court_carnival=/Game/FEL/Venues/VeniceBeach/VeniceBeach`
- DO NOT add skateboarding/snowboarding back until Phase 8 (staging promotion) when their maps are in MapsToCook

1.5 Fix `UnrealStarter/BasketballGame/Content/FEL/Config/ArenaSettings.json`

- SPLIT `"karate"` entry into two:

```
json
  "karate_h2h": {
    "unrealOpenLevelPackage": "/Game/FEL/Venues/Dojo/Dojo.Dojo",
    "modeDisplayName": "Karate · 1v1",
    "unrealBasketballSlice": "StreetBall",
    "targetScore": 5, "timeLimitSeconds": 150, "ballCount": 1
  },
  "karate_endless": {
    "unrealOpenLevelPackage": "/Game/FEL/Venues/Dojo/Dojo.Dojo",
    "modeDisplayName": "Karate · Endless",
    "unrealBasketballSlice": "StreetBall",
    "targetScore": 0, "timeLimitSeconds": 0, "ballCount": 1,
    "bScoringEnabled": true
  }
```

- ADD `who_scene_it` entry:

```
json
  "who_scene_it": {
    "unrealOpenLevelPackage": "/Game/FEL/Venues/NeuroArena/NeuroArena.NeuroArena",
    "modeDisplayName": "Who Scene It",
    "unrealBasketballSlice": "Practice",
    "ballSpawnType": "None", "ballCount": 0,
    "targetScore": 0, "timeLimitSeconds": 120,
    "bScoringEnabled": false
  }
```

- ADD `court_carnival` entry:

```
json
  "court_carnival": {
    "unrealOpenLevelPackage": "/Game/FEL/Venues/VeniceBeach/VeniceBeach.VeniceBeach",
```

```
    "modeDisplayName": "Court Carnival",
    "unrealBasketballSlice": "Practice",
    "ballSpawnType": "None", "ballCount": 0,
    "targetScore": 0, "timeLimitSeconds": 0,
    "bScoringEnabled": false
  }
```

1.6 Fix `UnrealIntegration/Source/FinalEvolutionLab/FELEmergentDeepLinkSubsystem.cpp`

- In `GetModeToVenueMap()` : rename the `"mario_party_fever"` key to `"court_carnival"`
- Verify all 19 mode_id → venue_token mappings match the list in step 1.1

1.7 Fix VenueRegistry files

- `backend/FEL_VenueRegistry.production.json` : Add `karate_endless` to Dojo venue's `game_modes` array. Add `who_scene_it` to Neuro_Arena. Add `court_carnival` to Venice_Beach_Court. Add `market_browse` to Sovereign_Shop (if missing).
- `UnrealStarter/BasketballGame/Config/FEL_VenueRegistry.production.json` : Add analytics entries for `karate_endless` , `who_scene_it` , `court_carnival` , `market_browse` .

QUALITY GATE 1

```
☐ FEL_ModeManager.production.json has exactly 19 entries in mode_registry
☐ FEL_ModeManager.production.json total_modes == 19
☐ GameMode.swift GameModeId enum has exactly 19 cases
☐ ue_mode_maps.json has exactly 19 entries in mode_to_unreal_map
☐ DefaultGame.ini [EmergentPlayMap] has exactly 18 entries (19 minus skateboarding/sno
wboarding removed + who_scene_it/court_carnival added — net 18; scene_it handled in C+
+ hardcode)
☐ ArenaSettings.json has karate_h2h AND karate_endless as separate entries (not uni-
fied "karate")
☐ ArenaSettings.json has who_scene_it and court_carnival entries
☐ No entry in [EmergentPlayMap] routes skateboarding or snowboarding to VeniceBeach
☐ who_scene_it status == "preview" in FEL_ModeManager
☐ court_carnival status == "preview" in FEL_ModeManager
☐ No reference to "mario_party_fever" remains anywhere in the codebase (grep -r "mario
_party_fever" returns 0)
☐ Build compiles (Swift + C++ — no missing enum cases in switch statements)
```

PHASE 2: ECONOMY COMPLETION

Goal: Wire shard rewards, PRQ delta, and XP cap into the backend session receipt pipeline.

2.1 Modify `backend/server.py` — `create_game_session` endpoint

Currently at approximately lines 502–508. After existing XP award logic, add:

Shard award:

```
outcome = data.get("outcome", "loss")  # "win" | "draw" | "loss"
combo = data.get("combo", 0)
criticals = data.get("criticals", 0)
shards_base = 50 if outcome == "win" else (25 if outcome == "draw" else 15)
combo_bonus = combo * 5 if combo > 3 else 0
critical_bonus = criticals * 10
shards_total = shards_base + combo_bonus + critical_bonus
await db.users.update_one({"user_id": user.user_id}, {"$inc": {"coins":
shards_total}})
await db.shard_transactions.insert_one({
    "user_id": user.user_id, "type": outcome,
    "amount": shards_total, "mode_id": s["mode_id"],
    "session_id": s["id"], "timestamp": datetime.utcnow()
})
```

PRQ delta:

```
mode_weights = {"basketball_h2h": 1.2, "basketball_dunk": 1.0, "basketball_3v3": 1.3,
    "karate_h2h": 1.4, "karate_endless": 1.4, "baseball": 1.0, "football": 1.5,
    "soccer": 1.1, "golf": 0.9, "tennis": 1.1, "volleyball": 1.2, "surfing": 1.05,
    "skateboarding": 1.0, "snowboarding": 1.0, "gymnastics": 1.0, "brain_brawl": 1.0}
prq_base = 2.0 if outcome == "win" else (0.5 if outcome == "draw" else 0.2)
weight = mode_weights.get(s["mode_id"], 1.0)
combo_b = min(1.0, combo * 0.05)
crit_b = min(0.5, criticals * 0.1)
score_diff = max(0, s["score"] - data.get("opponent_score", 0))
dom_b = min(0.5, score_diff * 0.05) if outcome == "win" else 0
prq_delta = min(100, prq_base * weight + combo_b + crit_b + dom_b)
await db.prq_metrics.update_one(
    {"user_id": user.user_id},
    {"$inc": {"composite_score": prq_delta}}, upsert=True
)
```

XP cap:

```
xp = min(500, max(10, s["score"] // 5))  # Cap at 500 per session
```

2.2 Add response fields

Return `shards_awarded`, `prq_delta`, and capped `xp_awarded` in the session response.

2.3 Create `shard_transactions` collection index

```
# In startup/ensure_indexes
await db.shard_transactions.create_index([("user_id", 1), ("timestamp", -1)])
```

QUALITY GATE 2

```
□ POST /api/games/session with outcome="win", score=100, combo=5, criticals=2 returns:
  - xp_awarded: 20 (100/5, under cap)
  - shards_awarded: 95 (50 + 5*5 + 2*10)
  - prq_delta: > 0
□ POST /api/games/session with score=5000 returns xp_awarded: 500 (cap enforced)
□ POST /api/games/session with outcome="loss" returns shards_awarded: 15
□ shard_transactions collection has new document after session
□ prq_metrics document updated for user
□ Existing tests still pass (backend/tests/)
```

PHASE 3: NEUROCOGNITIVE ENGINE — SWIFT LAYER

Goal: Build the Mental Resiliency Index (MRI) engine as a new Swift service.

3.1 Create `FinalEvolutionLab/Services/MentalResiliencyEngine.swift`

New file. Implements:

Tier A — Autonomic Recovery Velocity (ARV):

```swift
struct ARVState {
    var hrvBaseline: Double      // weeklyHRVAverage from HealthKitService
    var hrvStress: Double        // min HRV during peak intensity
    var hrvCurrent: Double       // latest HRV post-intensity
    var recoveryDeltaSec: Double // seconds to reach 90% baseline
    var score: Double            // 0–100
    static let maxRecoveryWindow: Double = 180.0
    static let recoveryThreshold: Double = 0.90
}
```

- Peak intensity detection: check `DynamicDifficulty.aggression >= 1.2` sustained 10s, OR `inputsPerSecond >= 4.0` for 8s, OR `GoldenEraComboEngine.chainLength >= 4`, OR `ArcadePhysics.neuralBurstActive`
- ARV calculation: `score = depression < 5 ? 95 : clamp(100 - (recoveryDelta / 180 * 100), 0, 100)`

Tier B — Executive Stamina Index (ESI):

```swift
struct ESIState {
    var csfBaseline: Double      // first-minute context-switch latency mean
    var csfCurrent: Double       // rolling 30s mean
    var csfScore: Double         // 0–100
    var eefErrorSlope: Double    // linear regression slope of errors per 60s window
    var eefScore: Double         // 0–100
    var ildBaseline: Double      // first-minute input latency mean (ms)
    var ildCurrent: Double       // rolling 30s mean
    var ildScore: Double         // 0–100
    var composite: Double        // 0.35*CSF + 0.40*EEF + 0.25*ILD
}
```

- CSF: track `AvatarStateMachine` state transitions → time to next valid input
- EEF: track `GoldenEraComboEngine.apexGrade == .miss` and shot misses per 60s windows

- ILD: track `InputManager` input registration latency vs session-start baseline

Tier C — Pacing Efficiency:

```
struct PacingState {
    var smartPauses: Int         // pacing events during strain (max 5)
    var missedPauses: Int        // strain windows without pacing
    var forcedRecoveries: Int    // health depleted / timeout events
    var score: Double            // 70 + smart*8 - missed*12 - forced*20
    var strainFlagged: Bool      // current strain state
}
```

- Strain detection: ESI < 50, or ARV < 50, or neuralReadinessGrade == .recovering, or DDA aggression >= 1.3
- Pacing event: voluntary pause, recovery mode selection, >50% input rate drop for 5s, sprint→idle transition for 3s

Composite MRI:

```
struct MentalResiliencyIndex {
    var arvScore: Double
    var esiComposite: Double
    var pacingScore: Double
    var mriScore: Double         // 0.30*ARV + 0.45*ESI + 0.25*Pacing
    var grade: MRIGrade          // UNBREAKABLE (≥85), RESILIENT (65–84), ADAPTING
(45–64), VULNERABLE (<45)
}
```

3.2 Wire MRI into existing systems

`FinalEvolutionLab/Models/ArcadePhysics.swift` — in `fromPRQ()`:
- Add parameter `mriARV: Double = 75.0`
- Modify `comboDecayRate`: multiply by ARV modifier (≥85: 0.85, 65–84: 0.95, 40–64: 1.0, <40: 1.15)

`FinalEvolutionLab/Models/InputManager.swift`:
- Add `esiModifier` property
- If ESI ≥ 80: `perfectGuardWindow *= 1.1`

`FinalEvolutionLab/Models/GoldenEraEngine.swift`:
- If ESI 40–59: `apexWindow *= 1.15` (compensatory widening)

`FinalEvolutionLab/Models/DynamicDifficulty.swift`:
- If ESI < 40: reduce `aiAggressionFloor` by 0.1

`FinalEvolutionLab/Utilities/PRQScoring.swift` — in `modeReward()`:
- Add parameter `mriScore: Double = 75.0`
- Add `mriBonus = (mriScore / 100.0) * 0.5` to return value

3.3 Feature flag

Add `static let enableNeurocognitiveEngine: Bool = true` to a config. When false, all modifiers default to 1.0 / 0.0 (no-op pass-through).

QUALITY GATE 3

```
□ MentalResiliencyEngine.swift compiles without errors
□ ARVState with recoveryDelta=60s produces score ~67 (100 - 60/180*100)
□ ARVState with recoveryDelta=0 produces score 90
□ ESIState with CSF=80, EEF=70, ILD=90 produces composite = 0.35*80 + 0.40*70 +
0.25*90 = 78.5
□ PacingState with 3 smart, 1 missed, 0 forced produces score = 70 + 24 - 12 - 0 = 82
□ MRI with ARV=78, ESI=78.5, Pacing=82 = 0.30*78 + 0.45*78.5 + 0.25*82 = 79.225
□ ArcadePhysics.fromPRQ() with mriARV=90 returns comboDecayRate * 0.85
□ PRQ.modeReward() with mriScore=80 adds 0.4 bonus
□ Feature flag false → all modifiers are 1.0 (no behavioral change)
□ All existing Swift tests pass
```

PHASE 4: NEUROCOGNITIVE ENGINE — UE BRIDGE & BACKEND

Goal: Wire MRI data through the UE bridge payload and backend storage.

4.1 Extend `FinalEvolutionLab/Models/SystemScanRecord.swift`

Add to `UnrealSystemScanPayload`:

```swift
struct NeuroCognitivePayload: Codable {
    let schemaVersion: Int = 1
    let mriScore: Double
    let mriGrade: String
    let tierA_ARV: Double
    let tierB_ESI: Double
    let tierC_Pacing: Double
    let strainFlagged: Bool
    let comboDecayModifier: Double    // 0.85–1.15
    let perfectGuardModifier: Double  // 1.0 or 1.1
    let qteWindowModifier: Double     // 1.0 or 1.15
    let ddaFloorOverride: Double      // -1.0 (no override) or actual floor
    let pacingSuggestion: String      // "none" | "recommend_pause" | "mandat-
ory_cooldown"
}
```

Deliver via existing `SystemScanFirestoreSync.deliverScanToBridge()` → `UnrealManager.deliverSystemScanJSON()`.

4.2 Add Firestore collection

Path: `users/{uid}/neurocognitive_sessions/{sessionId}`
Schema:

```
mri_score: number
mri_grade: string
tier_a: { arv_score, hrv_baseline, hrv_stress, recovery_delta_sec }
tier_b: { esi_composite, csf_score, eef_score, ild_score, decel_curve: [[time, esi]] }
tier_c: { pacing_score, smart_pauses, missed_pauses, forced_recoveries }
mode_id: string
session_duration_sec: number
created_at: Timestamp
```

Security rule: `allow read, create: if request.auth.uid == userId` (append-only, same as system_scans).

4.3 Add backend endpoints in `backend/server.py`

POST /api/neurocognitive/session:

```python
@api_router.post("/neurocognitive/session")
async def create_neurocognitive_session(data: dict, user = Depends(get_current_user)):
    doc = {
        "user_id": user.user_id,
        "session_id": data.get("session_id"),
        "mode_id": data.get("mode_id"),
        "mri_score": data.get("mri_score", 75.0),
        "mri_grade": data.get("mri_grade", "ADAPTING"),
        "tier_a": data.get("tier_a", {}),
        "tier_b": data.get("tier_b", {}),
        "tier_c": data.get("tier_c", {}),
        "session_duration_sec": data.get("session_duration_sec", 0),
        "created_at": datetime.utcnow()
    }
    await db.neurocognitive_sessions.insert_one(doc)
    return {"status": "ok", "mri_score": doc["mri_score"]}
```

GET /api/neurocognitive/history:

```python
@api_router.get("/neurocognitive/history")
async def get_neurocognitive_history(user = Depends(get_current_user)):
    cursor = db.neurocognitive_sessions.find(
        {"user_id": user.user_id}
    ).sort("created_at", -1).limit(30)
    sessions = await cursor.to_list(30)
    return {"sessions": [{k: v for k, v in s.items() if k != "_id"} for s in ses-
sions]}
```

GET /api/neurocognitive/baseline:

```
@api_router.get("/neurocognitive/baseline")
async def get_neurocognitive_baseline(user = Depends(get_current_user)):
    cursor = db.neurocognitive_sessions.find(
        {"user_id": user.user_id}
    ).sort("created_at", -1).limit(10)
    sessions = await cursor.to_list(10)
    if not sessions:
        return {"arv_avg": 75.0, "esi_avg": 75.0, "pacing_avg": 70.0, "mri_avg": 75.0}
    return {
        "arv_avg": sum(s.get("tier_a", {}).get("arv_score", 75) for s in sessions) / l
en(sessions),
        "esi_avg": sum(s.get("tier_b", {}).get("esi_composite", 75) for s in ses-
sions) / len(sessions),
        "pacing_avg": sum(s.get("tier_c", {}).get("pacing_score", 70) for s in ses-
sions) / len(sessions),
        "mri_avg": sum(s.get("mri_score", 75) for s in sessions) / len(sessions),
    }
```

4.4 Add MongoDB indexes

```
await db.neurocognitive_sessions.create_index([("user_id", 1), ("created_at", -1)])
```

QUALITY GATE 4

```
□ POST /api/neurocognitive/session with valid MRI data returns 200
□ GET /api/neurocognitive/history returns array (empty if first call)
□ GET /api/neurocognitive/baseline returns averages
□ Firestore security rules allow user to create under their own UID
□ UnrealSystemScanPayload includes neurocognitive block
□ deliverScanToBridge sends neurocognitive data to UE framework
□ All existing backend tests pass
```

PHASE 5: SESSION RECEIPT INTEGRATION

Goal: Ensure all 12 production modes produce complete session receipts with XP + shards + PRQ delta + MRI data.

5.1 Extend `create_game_session` response

Add `neurocognitive` block to the session response:

```
response["neurocognitive"] = {
    "mri_score": data.get("mri_score", 75.0),
    "mri_grade": data.get("mri_grade", "ADAPTING"),
    "tier_a_arv": data.get("tier_a_arv", 75.0),
    "tier_b_esi": data.get("tier_b_esi", 75.0),
    "tier_c_pacing": data.get("tier_c_pacing", 70.0)
}
```

5.2 Add shard pacing bonus

If `tier_c_pacing >= 75`: add 5% shard bonus

```
pacing_score = data.get("tier_c_pacing", 70.0)
pacing_bonus = int(math.ceil(shards_total * 0.05)) if pacing_score >= 75 else 0
shards_awarded = shards_total + pacing_bonus
```

5.3 Add MRI bonus to PRQ delta

```
mri_score = data.get("mri_score", 75.0)
mri_bonus = (mri_score / 100.0) * 0.5
prq_delta = min(100, prq_base * weight + combo_b + crit_b + dom_b + mri_bonus)
```

5.4 Integrate brain_brawl with standard receipt

In `submit_bb` endpoint (POST /api/brain-brawl/submit), also write to `game_sessions` collection with `mode_id: "brain_brawl"` so it appears in standard session history. Keep the dedicated `brain_brawl_sessions` collection for backwards compatibility.

5.5 Rename mario-party endpoints

- `GET /api/games/mario-party` → `GET /api/games/court-carnival` (add alias for backwards compat)
- `POST /api/games/mario-party/session` → `POST /api/games/court-carnival/session` (add alias)

QUALITY GATE 5

```
□ All 12 production modes (basketball_h2h through surfing) produce complete session
receipts with: xp_awarded, shards_awarded, prq_delta, neurocognitive block
□ brain_brawl submit also creates game_sessions entry
□ Pacing bonus: session with tier_c_pacing=80, outcome=win → shards = 50 + bonuses +
5% pacing
□ MRI bonus: session with mri_score=90 → prq_delta includes +0.45 bonus
□ /api/games/court-carnival returns valid config
□ No endpoint references "mario-party" without alias redirect
□ All backend tests pass
```

PHASE 6: HUD & OVERLAY IMPLEMENTATION

Goal: Build the 6 HUD components in the WKWebView overlay React SPA.

6.1 Scoreboard Panel

- File: `frontend/src/components/hud/Scoreboard.tsx`
- Position: top-center, 10% from top, 60% viewport width
- Shows: player score, mode display name, opponent score, time elapsed, venue name
- Data source: WebSocket events from EmergentBridge (`match_score`, `session_state`)

6.2 Active Statistical Ticker

- File: `frontend/src/components/hud/StatTicker.tsx`
- Position: top-right, 20% width
- Shows: PRQ (with grade), MRI (with grade), ARV, ESI, XP counter, shard counter
- Data source: `sovereign_telemetry` WebSocket events (0.1s tick), session receipt for XP/shards
- Colors: PRQ=#00E5FF, MRI=#A855F7, ARV=#22C55E, ESI=#F59E0B

6.3 Takeover Meter

- File: `frontend/src/components/hud/TakeoverMeter.tsx`
- Position: left edge, vertical bar, 40% viewport height
- Shows: NeuralDrive 0–100 as fill bar with aura color gradient
- Glow: 4px bloom at fill level
- Data source: `sovereign_telemetry` neural_drive field

6.4 Combo Multiplier / Focus Streak

- File: `frontend/src/components/hud/ComboStreak.tsx`
- Position: floating center-right (70% X, 40% Y)
- Shows: combo multiplier (×1.0–×4.0), 6 focus streak pips, last QTE grade flash
- Animation: punch-scale on increment, shake on MISS, gold burst on PERFECT
- Data source: `combo_update` WebSocket events

6.5 Neurocognitive Overlay

- File: `frontend/src/components/hud/NeuroOverlay.tsx`
- Position: bottom-left, 25% width
- Shows: MRI bar (0–100), CSF/EEF/ILD dots (green ≥60, yellow 40–59, red <40), pacing grade
- Visibility: fades in during active session, auto-hides in menus
- Data source: `sovereign_telemetry` neurocognitive block

6.6 Recovery Prompt

- File: `frontend/src/components/hud/RecoveryPrompt.tsx`
- Trigger: ESI < 40 OR Pacing < 25
- Shows: “Recovery Recommended” with Continue / Pause buttons
- Max frequency: once per 120s
- Border: 2px #FF3366

6.7 Wire into existing overlay

Import all 6 components into the existing WKWebView overlay layout. Gate behind `ENABLE_NEUROCOGNITIVE_ENGINE` feature flag in frontend config.

QUALITY GATE 6

```
□ All 6 HUD components render without errors
□ Scoreboard updates when WebSocket match_score fires
□ StatTicker shows PRQ and MRI values from telemetry
□ Takeover Meter fills/drains with neural drive changes
□ ComboStreak animates on combo_update events
□ NeuroOverlay shows correct dot colors for CSF/EEF/ILD
□ RecoveryPrompt appears when ESI < 40 (mock data test)
□ RecoveryPrompt does NOT appear more than once per 120s
□ Feature flag false → no neurocognitive HUD elements render
□ No layout overlap with existing dashboard elements
```

PHASE 7: UE 5.7 C++ SUBSYSTEM

Goal: Create the NeuroCognitive subsystem in the UE C++ layer.

7.1 Create header: `UnrealIntegration/Source/FinalEvolutionLab/FELNeuroCognitiveSubsystem.h`

```
UCLASS()
class UFELNeuroCognitiveSubsystem : public UGameInstanceSubsystem
{
    GENERATED_BODY()
public:
    virtual void Initialize(FSubsystemCollectionBase& Collection) override;

    UFUNCTION(BlueprintCallable) float GetMRIScore() const;
    UFUNCTION(BlueprintCallable) float GetComboDecayModifier() const;
    UFUNCTION(BlueprintCallable) float GetPerfectGuardModifier() const;
    UFUNCTION(BlueprintCallable) float GetQTEWindowModifier() const;
    UFUNCTION(BlueprintCallable) float GetDDAFloorOverride() const;
    UFUNCTION(BlueprintCallable) bool IsStrainFlagged() const;
    UFUNCTION(BlueprintCallable) FString GetPacingSuggestion() const;

    void UpdateFromBridgePayload(const TSharedPtr<FJsonObject>& NeuroCognitiveJson);

private:
    float MRIScore = 75.0f;
    float ComboDecayModifier = 1.0f;
    float PerfectGuardModifier = 1.0f;
    float QTEWindowModifier = 1.0f;
    float DDAFloorOverride = -1.0f;
    bool bStrainFlagged = false;
    FString PacingSuggestion = TEXT("none");
};
```

7.2 Create source: `FELNeuroCognitiveSubsystem.cpp`

- Parse neurocognitive block from bridge JSON
- Expose BlueprintCallable getters for BP gameplay logic
- Default all values to no-op when neurocognitive block is missing

7.3 Wire into bridge

In `UFELEmergentBridgeSubsystem`, when receiving system scan JSON, check for `"neurocognitive"` key and call `UFELNeuroCognitiveSubsystem::UpdateFromBridgePayload()`.

7.4 Add to Build.cs

No new module dependencies needed (Json already included).

QUALITY GATE 7

```
□ FELNeuroCognitiveSubsystem.h/.cpp compile without errors
□ Subsystem initializes with default values (MRI=75, all modifiers=1.0)
□ UpdateFromBridgePayload correctly parses neurocognitive JSON
□ Blueprint nodes accessible: GetMRIScore, GetComboDecayModifier, etc.
□ Missing neurocognitive block in JSON → defaults retained (no crash)
□ Full UE C++ build passes (iOS + Linux targets)
```

PHASE 8: STAGING MODE PROMOTION PREPARATION

Goal: Prepare staging modes (gymnastics, brain_brawl) for production promotion. Fix skateboarding/snowboarding routing.

8.1 Gymnastics → production readiness

- Add PRQ modeWeight to `PRQScoring.swift`: `case .gymnastics: return 1.0`
- Add to `DynamicDifficulty.swift` mode-specific DDA window scale
- Verify TrainingFloor map loads via deep link
- Verify session receipt posts correctly

8.2 Brain Brawl → production readiness

- Integrate `submit_bb` with standard `create_game_session` (dual-write)
- Add shard rewards to brain_brawl sessions (using loss formula: 15 + bonuses since competitive scoring is different)
- Add PRQ modeWeight: `case .brainBrawl: return 1.0`
- Add PRQ delta computation for brain_brawl completions

8.3 Skateboarding/Snowboarding routing fix

- DO NOT add maps to MapsToCook yet (maps don't exist as dedicated assets)
- Ensure [EmergentPlayMap] has no entries for skateboarding/snowboarding (removed in Phase 1)
- Add staging gate: in `backend/server.py` `launch_stream_mode`, check mode status in ModeManager — reject launch if status != "production"

```
python
  mode_entry = MODE_MANAGER["mode_registry"].get(mode_id)
  if not mode_entry or mode_entry.get("status") != "production":
      return {"error": f"Mode {mode_id} is not available", "status": "staging"}
```

8.4 Promote gymnastics and brain_brawl

- Change status in FEL_ModeManager.production.json:
- `gymnastics`: "staging" → "production"
- `brain_brawl`: "staging" → "production"
- Update `production_modes` count: 12 → 14

QUALITY GATE 8

```
☐ POST /api/streaming/launch-mode with mode_id="skateboarding" returns error (staging
gate)
☐ POST /api/streaming/launch-mode with mode_id="gymnastics" succeeds (now production)
☐ brain_brawl submit creates both brain_brawl_sessions AND game_sessions entries
☐ brain_brawl session receipt includes shards_awarded > 0
☐ PRQ.modeWeight(.gymnastics) returns 1.0
☐ PRQ.modeWeight(.brainBrawl) returns 1.0
☐ FEL_ModeManager production_modes == 14
☐ Deep link → gymnastics → TrainingFloor loads → session receipt → economy
☐ Deep link → brain_brawl → NeuroArena loads → quiz → session receipt → economy
```

PHASE 9: SMOKE TEST MATRIX

Goal: Execute the full acceptance test suite across all 14 production modes.

9.1 Universal tests (run for each of 14 production modes)

For each mode in [basketball_h2h, basketball_dunk, basketball_3v3, karate_h2h, karate_endless, baseball, football, soccer, golf, tennis, volleyball, surfing, gymnastics, brain_brawl]:

#	Test	Pass Criteria
T1	Deep link launch	`finalevolution://launch?mode={mode_id}` → correct UE map opens
T2	MapLoaded event	WebSocket `map_loaded` fires within 10 seconds
T3	Session receipt	`POST /api/games/session` returns 200 with valid doc
T4	XP award	User XP incremented, ≤ 500 cap
T5	Shard reward	Correct shards (50/25/15 + bonuses) credited
T6	PRQ delta	prq_delta > 0 with correct mode weight
T7	Activity feed	Entry with type=game in activity_feed collection
T8	Sovereign telemetry	sovereign_telemetry JSON emitted with correct arena_game_mode_id
T9	MRI data	neurocognitive block present in session receipt (or defaults if engine disabled)
T10	Input scheme	Mode-correct input (charge/swipe/etc.) registers

9.2 Mode-specific tests

Mode	Specific Test
basketball_h2h	Charge → shot → basket → score increments → win at 3
basketball_dunk	Charge jump → dunk → style score → win at 21
basketball_3v3	3v3 team → 2 balls active → score to 11
karate_h2h	Strike → combo → score to 5 or time 150s
karate_endless	Waves spawn → defeat → harder → death → score submitted
baseball	Swipe → bat → distance → HR → score to 6 or time 300s
football	Kick return → dodge → TD → score to 3
soccer	Swipe kick → goal/save → alternate → score to 5
golf	SwipeGolf → flight → pin distance → 5 holes
tennis	Serve → rally → point → score to 5 or time 120s
volleyball	Set → spike → point → score to 5 or time 120s
surfing	Wave → rhythm tricks → score → time 180s
gymnastics	Routine → form score → 5 routines
brain_brawl	Questions → answers → XP (score//10) + shards

9.3 HUD tests

Test	Pass Criteria
H1	Scoreboard shows correct mode name and scores
H2	StatTicker shows PRQ and MRI with correct grades
H3	Takeover Meter fills to match neural drive
H4	ComboStreak animates on combo events
H5	NeuroOverlay CSF/EEF/ILD dots correct colors
H6	RecoveryPrompt fires when ESI < 40
H7	RecoveryPrompt respects 120s cooldown

9.4 Negative tests

Test	Pass Criteria
N1	Deep link to “skateboarding” → rejected (staging)
N2	Deep link to “who_scene_it” → rejected (preview)
N3	Deep link to “court_carnival” → rejected (preview)
N4	Session with score=999999 → XP capped at 500
N5	Neurocognitive engine disabled → all modifiers 1.0, no MRI HUD

QUALITY GATE 9

```
□ All 14 production modes pass all 10 universal tests (140 test cases)
□ All 14 mode-specific tests pass
□ All 7 HUD tests pass
□ All 5 negative tests pass
□ Total: 166 test cases, 0 failures
□ Smoke test results documented in docs/smoke_test_results.md
```

PHASE 10: SHIP READINESS & FINAL VALIDATION

Goal: Final pre-ship checklist. Verify every “do not ship until” gate.

10.1 CI Alignment Check

```
./infra/fel_prebuild_ci_check.sh --strict
# Must pass all 6 points:
# 1. .uproject filename = FinalEvolutionLab
# 2. Target.cs class name matches
# 3. DefaultGame.ini BundleIdentifier = com.finalevolutionlab.sovereign
# 4. Info.plist UE_PROJECT_NAME = FinalEvolutionLab (case-sensitive)
# 5. Directory paths match
# 6. [Emergent] config section present
```

10.2 Registry Consistency Audit

```
□ FEL_ModeManager.production.json: 19 entries, total_modes=19, production_modes=14
□ GameMode.swift: 19 enum cases
□ ue_mode_maps.json: 19 entries
□ ArenaSettings.json: 19 mode entries (17 original + who_scene_it + court_carnival)
□ DefaultGame.ini [EmergentPlayMap]: 18 entries (no skateboarding/snowboarding)
□ GetModeToVenueMap() in C++: 19 entries (court_carnival, not mario_party_fever)
□ All map paths use /Game/FEL/Venues/{Name}/{Name} format
□ Zero references to "mario_party_fever" in codebase
□ who_scene_it status = "preview"
□ court_carnival status = "preview"
```

10.3 Economy Audit

```
□ create_game_session awards XP (capped 500), shards, PRQ delta, MRI bonus
□ shard_transactions collection indexed and writing
□ prq_metrics updates on every session completion
□ neurocognitive_sessions writing on every session completion
□ Pacing bonus (+5% shards when Tier C ≥ 75) functional
```

10.4 Neurocognitive Engine Audit

```
□ MentalResiliencyEngine.swift compiles and runs
□ Tier A (ARV) calculates from HRV data
□ Tier B (ESI) tracks CSF, EEF, ILD passively
□ Tier C (Pacing) detects smart pauses during strain
□ MRI composite feeds into ArcadePhysics, DDA, GoldenEra, PRQ delta
□ UE bridge payload includes neurocognitive block
□ FELNeuroCognitiveSubsystem exposes Blueprint-callable getters
□ Feature flag disables entire engine cleanly
```

10.5 Build & Deploy

- [ ] iOS Shipping build: ./fel_ue5_ios_shipping_package.sh --full-cook --shipping --export-ipa
- [ ] .app bundle contains cookeddata/.pak (descriptor-safe)
- [ ] CFBundleIdentifier = com.finalevolutionlab.sovereign
- [ ] HealthKit usage strings in Info.plist
- [ ] finalevolution:// URL scheme registered
- [ ] Upload to App Store Connect / TestFlight
- [ ] Write superapp release metadata: ./scripts/write_superapp_release_metadata.sh
- [ ] No AltStore/SideStore/OTA references in codebase

10.6 E3DS Deployment

- [ ] Linux build: ./fel_ue5_eagle3d_linux_package.sh
- [ ] Pulumi deploy: ./infra/deploy_e3ds.sh
- [ ] Pixel Streaming connects and resolves all 14 production maps
- [ ] backend/.env updated with E3DS_STREAM_URL

QUALITY GATE 10 (FINAL)

- [ ] fel_prebuild_ci_check.sh --strict passes ALL 6 points
- [ ] All 14 production modes launch, play, and receipt correctly
- [ ] All 4 staging modes are blocked from launch (staging gate)
- [ ] All 2 preview modes are blocked from launch
- [ ] market_browse loads as non-game module (no session receipt)
- [ ] Full economy loop works: play → XP + shards + PRQ + MRI
- [ ] Neurocognitive HUD renders during gameplay
- [ ] Recovery prompt fires on ESI < 40
- [ ] iOS TestFlight build installs and runs on device
- [ ] E3DS Pixel Streaming resolves all production maps
- [ ] docs/smoke_test_results.md documents all 166 test results
- [ ] No P0 issues remain

PHASE SUMMARY

Phase	Goal	Files Touched	Quality Gate
1	Registry alignment	7 config/source files	12 checks
2	Economy completion	server.py	6 checks
3	Neuro engine — Swift	6 Swift files + 1 new	10 checks
4	Neuro engine — UE/ Backend	SystemScanRecord, server.py, Firestore	7 checks
5	Session receipt integ- ration	server.py, brain_brawl, court_carnival	7 checks
6	HUD & overlay	6 new React compon- ents	10 checks
7	UE C++ subsystem	2 new C++ files	6 checks
8	Staging promotion	Swift scoring, serv- er.py, ModeManager	9 checks
9	Smoke test matrix	docs/	166 test cases
10	Ship readiness	Build scripts, deploy	12 checks

Total: 245 quality gate checks across 10 phases.

Seele AI Execution Directive v1.0
Repository: Final-Evolution-Lab (anti-gravity-fel)
Date: 2026-05-22